DEVELOPER & DESIGNER GUIDE

The AIDD Agent System

AI-Augmented Discovery and Design: a gate-controlled process for domain-heavy products

How the system works, what every agent and skill does, how to run a project through it end to end, and how to extend it. Written for the designer running the process, the developer receiving its output, and anyone maintaining the system itself.

14	34	9	3
AGENTS	SKILLS	COMMANDS	GATES

Version 2.0 · Built to the Agent Skills open standard · 31 July 2026

Contents

How to read this guide 4

Part 1 · Orientation 5

1.1 What this system is 5
1.2 The five ideas it is built on 5
1.3 The process at a glance 6
1.4 Quick start 7
1.5 Repository layout 7

Part 2 · Core mechanics 9

2.1 Provenance: tags and confidence 9
2.2 The verification ladder 10
2.3 Contradictions are deliverables 10
2.4 Gates 10
2.5 Corrections and the rework machinery 11
2.6 Continuous conformance 12
2.7 Working without a spec 14
2.8 The five adaptation variables 15
2.9 The design floor 16

Part 3 · The phases, step by step 17

3.1 Phase 0: Intake 17
3.2 Phase 1: Empathize and Define 17
3.3 Phase 2A: Initial Design 19
3.4 Gate 1 20
3.5 Phase 2B: Further Design 20
3.6 Gate 2 23
3.7 Phase 3: Test and Prototype 23
3.8 Gate 3 24
3.9 Handoff and the md-spec loop 24

Part 4 · The agents 27

4.1 How agents, skills and the orchestrator relate 27
4.2 The routing table 28
4.3 Agent cards 29

Part 5 · The skills 36

5.1 Skill anatomy 36
5.2 Intake and discovery 37
5.3 Research, knowledge and prototyping 38
5.4 Convergence 40
5.5 Production intent 42

5.6 Quality, gates and close 44

Part 6 · Commands 45

Part 7 · Artifacts and templates 46

Part 8 · Reference 48

8.1 Artifact dependency order 48
8.2 Guardrails in summary 48
8.3 Troubleshooting 50
8.4 Adapting the process to the project 51
8.5 Extending the system 51
8.6 Glossary of system terms 52

BEFORE YOU START

How to read this guide

This guide is a reference, not a novel. Nobody needs all of it. Find your row below, read those parts, and come back to the rest when the project reaches it.

You are	Read
Running a project through the system for the first time	Part 1 (orientation), Part 2 (core mechanics), then Part 3 phase by phase as you reach each one. Keep Part 6 (commands) open in a tab.
A designer joining a project mid-flight	Part 1.3 (the process at a glance), Part 2.1 and 2.5 (provenance and corrections), then the agent cards in Part 4 for whichever role you are covering, and the skills those agents load in Part 5.
A developer receiving a handoff	Part 2.1 (what the tags mean), Part 3.9 (what handoff contains and how the walkthrough runs), Part 7 (the artifacts you are being given), and Part 9.2 (the routing rule for build-time questions). Roughly twenty minutes.
A product owner or client-side reader	Part 1, Part 2.4 (what gates ask of you), and Part 2.7 (what happens when there is no spec). The rest is machinery you are welcome to but do not need.
Extending or maintaining the system	Part 2 in full, Part 5 (skill anatomy), and Part 9.5 (extension rules). The system is plain markdown throughout; nothing is compiled.

> CONVENTIONS IN THIS GUIDE
>
> `monospace` marks a file path, a command, or a literal value. **PHASE** chips mark which phase a thing belongs to. Boxed rules in red are non-negotiable constraints from `GUARDRAILS.md`; boxed notes in amber are the failure modes teams actually hit.

PART 1

Orientation

What this system is, the ideas it rests on, the shape of the process, and how to start a project in the next ten minutes.

1.1 What this system is

An agentic implementation of the AI-Augmented Discovery and Design process: a three-phase, gate-controlled product design workflow built for domain-heavy projects, where the cost of misunderstanding the client's rules is far higher than the cost of drawing the wrong screen.

You talk to the orchestrator, which is the main session. It routes work to specialist agents. Agents load skills. Skills carry the craft: the procedures, the decision tables, the probing banks, the failure modes. Nothing advances past a gate without that gate's exit criteria met and its report filed, and a standing conformance engine continuously checks that every artifact still agrees with the documents it derives from.

Everything is plain markdown. It can be dropped into a repository and used directly with Claude Code, or read by any agent runtime that understands markdown instructions, or used by humans with no agent at all: the templates, the provenance tags and the gate reports work identically.

1.2 The five ideas it is built on

1. Provenance is the load-bearing wall

Every claim about the client's world carries a source tag and a confidence marker. Most failure modes of a discovery process are attribution collapse: six weeks in, nobody can say whether "the system must lock the record after posting" came from the domain authority, from the anchor document, from an industry convention, or from a model's plausible guess. Once that distinction is lost it cannot be recovered, and every downstream decision inherits an unknown risk.

2. Gates are the only thing that stops work

Three formal gates, each with required attendees, written exit criteria and a filed report. A gate held without its required attendees is recorded as NOT HELD, whatever happened in the room. This strictness is the point: a process whose checkpoints can be waved through has checkpoints that mean nothing.

3. The prototype is an interview instrument, not a preview

Phase 2A produces a throwaway prototype whose success is measured in *corrections produced*, not approval received. It is built to be wrong in a useful way, framed that way out loud, and archived immediately after Gate 1. Hi-fi never starts from its screens.

4. One document reconciles everything

The refined requirement document does not restate the client's spec; it shows where the spec and the domain authority disagreed and why the resolution won. Its rationale column is the answer to "why does this rule work this

way" for the life of the product. Every structural correction, from any source and at any time, passes through it before any artifact is edited.

5. The loop pays the next project

At close, a developer and a designer review three shared specification files against what the project actually built. Those files are consumed by the next project's prototype prompts, so generated output matches real team conventions and can be re-used rather than referenced. It is the only step that pays out later rather than now, which makes it the easiest to skip and the most expensive to have skipped.

1.3 The process at a glance

```
PHASE 0   Intake            Classify what actually arrived. Fill the spec slot.
                              Open the assumption register.
                                   |
PHASE 1   Empathize + Define  Anchor analysis, glossary, questions, the D&D call,
                              contradiction mining.
   GATE                       Written confirmation of the problem set by the DA.
                                   |
PHASE 2A  Initial Design      Parallel: research, knowledge base, as-is flows,
                              style seed, prompts, throwaway prototype.
   GATE 1                     DA + PO. The set reviewed together, never piecemeal.
                              Heavy correction = success. Prototype archived.
                                   |
PHASE 2B  Further Design      Refined requirements -> ERD -> to-be flows ->
                              breakdown -> flow optimization -> IA -> userflows
                              -> prioritized feature list (signed).
   GATE 2                     DA + PO + developer. Model first, screens last.
                              List locks on fixed-bid engagements.
                                   |
PHASE 3   Test + Prototype    Hi-fi specs for prioritized features only:
                              every state, dev notes, interactions, status
                              mapping, real copy, realistic content.
   GATE 3                     Internal, line by line against requirements.
                              Then client validation.
                                   |
HANDOFF   Transfer + Loop     Full artifact set, live walkthrough with the dev
                              team, md-spec loop updating shared-specs and
                              recording the three cross-project metrics.
```

Two properties of this shape are worth naming. First, the Test stage re-opens the problem space: corrections found late are routed *backward* through the requirement document, never absorbed silently into a screen. Second, the phases are deliberately front-loaded. Phase 3 designs only what the prioritized list contains, and that narrowing is what pays for the investment in Phases 1 and 2.

1.4 Quick start

Starting a new project

```
# in the repository root, with Claude Code running
/new-project acme-invoicing
```

The orchestrator copies `workspace/_TEMPLATE/`, then asks for the five adaptation variables plus two the design layer needs (accessibility obligation, and device and environment per role). Answer honestly; they change how the whole process routes. Do not guess the domain authority: if nobody knows who can rule on correctness, that is the first thing to find out, and Phase 1 blocks until you do.

Immediately after, intake runs:

```
/intake-spec
```

Give it whatever you have: a PRD, a gap document, a slide deck, a login to the old system, or nothing at all. It classifies what arrived, audits it against fourteen dimensions, fills the spec slot at `00-inputs/PRD.md`, and starts a substitute path if there is no real spec.

Joining a project already running

```
/phase-status
```

The standup view: current phase, open blockers, next gate and its unmet exit criteria, stale artifacts with the dependency that staled them, the conformance headline, and the next three concrete actions with their owning agents.

> **THE ONE HABIT THAT MATTERS MOST**
>
> When you paste notes, forward an email, or relay a call, **say who said what**. The system can do nothing honest with "the client wants X". It can do a great deal with "[DA] said X on 14 March, and [DOC:4.2] says the opposite".

1.5 Repository layout

```
CLAUDE.md            orchestrator: state machine, routing, gates, rework, conformance cadence
GUARDRAILS.md        global rules; every agent loads this; overrides everything else
docs/                USAGE.md, SKILL-INDEX.md, and this guide
config/              PROJECT_CONFIG and STATE templates, copied per project
.claude/agents/      14 subagents, one file per role
.claude/commands/    9 slash commands
.claude/skills/      34 skills, one folder each, SKILL.md plus references/ for depth
templates/           16 artifact templates the skills reference
shared-specs/        design-system / EngineeringSpec / CodeReadability (cross-project)
workspace/           _TEMPLATE/ skeleton with placeholders, plus one folder per project
```

Inside a project workspace, folders are numbered in the order the process fills them, so the directory listing is itself a progress indicator:

```
workspace/{project}/
  PROJECT_CONFIG.md   the adaptation variables; routing reads this
  STATE.md            live ledger: phase, gates, freshness, rework log, risks, conformance
  00-inputs/          PRD.md (the spec slot, never absent), spec-intake.md, assumptions.md,
                      audit/, client materials (read-only once filed)
  10-discovery/       anchor-analysis, glossary, questions, transcripts/, problem-statements,
                      solution-hypotheses, open-questions, derived-brief
  20-research/        incumbents/, research-notes/, kb-index.md, client/ authority/ external/
  30-design-language/ style-guide.md, skills-md/, prompts/
  40-models/          as-is/, to-be/, flow-optimization/, erd.md, process-breakdown.md,
                      ia.md, ia-validation.md, userflows/, sitemap.md
  50-prototype/       prompt-used.md, build-notes.md, the build; ARCHIVED/ after Gate 1
  60-requirements/    prd-draft.md, refined-requirements.md, prioritized-features.md,
                      traceability.md
  70-hifi/            {feature}/spec.md, {feature}/content.md, status-mapping.md, critiques/
  80-gates/           gate reports, pre-gate critiques
  90-handoff/         manifest.md, walkthrough-notes.md
  conformance/        dated conformance snapshots
```

PART 2

Core mechanics

Nine mechanisms do the actual work of keeping this process honest. Everything in Parts 3 through 7 is an application of one or more of them. This is the part worth reading closely.

2.1 Provenance: tags and confidence

Any statement that describes the client's world (a problem, a rule, a process step, a status, a permission, a preference) carries a source tag and a confidence marker wherever it is written down. Both, always, in every artifact.

Source tag	Meaning
`[PO]`	The product owner said it
`[DA]`	The domain authority said it
`[TEAM:name]`	A team member raised it, named
`[DOC:section]`	The anchor document, with its section reference
`[USER:role]`	An end user, by role, from an interview or observation
`[RESEARCH:source]`	External research, with the source named
`[LLM]`	Model inference, not yet verified by any human

Confidence	Meaning
`(confirmed)`	The domain authority has confirmed it, in writing or on a recording
`(asserted)`	A human said it once, unverified by the authority
`(inferred)`	Derived by reasoning or by a model; no human has said it

> **HARD RULES**
>
> A domain rule with only `[LLM]` provenance never enters a design artifact; it goes to `open-questions.md` for verification. `[RESEARCH]` informs design decisions but never sets a client rule: industry convention is not the client's reality. When two tagged statements conflict, do not pick a winner; record the conflict. If you catch yourself writing an untagged claim into an artifact, stop and tag it; if you cannot tag it, it does not go in.

Why the distinction between (asserted) and (confirmed) is worth the friction

A single human saying something once is evidence that it might be true. The domain authority confirming it in writing is evidence that the project can be held to it. Those are different things, and the gap between them is where

most late-discovered requirement defects live. The system's answer is mechanical: behaviour-constraining claims (a rule, a status, a permission, a lifecycle property) must be `(confirmed)` before Gate 2. Everything still `(asserted)` sits in a visible open-items list with an owner, and open items block the gate. Visibility is the mechanism; nobody has to remember.

2.2 The verification ladder

Three rungs, and each artifact type is allowed only certain rungs.

Rung	Allowed in
`[LLM] (inferred)`	open-questions, research notes, draft prompts. Nowhere else, ever.
`(asserted)`	Problem statements, tentative artifacts, the provisional PRD draft.
`(confirmed)`	Refined requirements, ERD constraints, status mapping: anything a developer will build against.

2.3 Contradictions are deliverables

Consolidation that smooths disagreement away destroys the most valuable output of discovery. When the anchor document says one thing and the domain authority says another, that gap is the finding. It is surfaced, tagged on both sides, given an owner and a due date, and pushed into `STATE.md`. Unowned contradictions block the Phase 1 exit.

This has a concrete consequence for how models are used. The consolidation instruction is always "*list the contradictions between these sources*". The verbs merge, reconcile and harmonize are banned from the instruction, because a fluent model asked to reconcile will produce a smooth, plausible, single narrative, and the disagreement it dissolved was the thing you were paying to find.

Contradictions are typed, because the type predicts who must resolve it and how urgent it is:

Type	Pattern	Resolution path
FACT	Different values (daily vs weekly, 30 vs 45 days)	Cheap; one is stale. DA rules, often same week
RULE	Disagreement on what is allowed or required	Expensive and load-bearing; blocks dependent ERD work
SCOPE	Disagreement on what the project covers	PO-owned; unresolved, it poisons prioritization
PROCESS	Different sequences for the same work	Often BOTH true: map the variants, do not pick
VOCABULARY	Same word, different meanings by faction	Glossary collision; route with an owner
AUTHORITY	Disagreement on who decides or overrides	Politically hot; handle in a small setting, not a gate
DOC-VS-REALITY	The document commits to something the room denies	The classic finding; usually "document is old", but confirm

2.4 Gates

A gate produces exactly three things: a held-or-not-held fact, a classified correction list, and a filed report. Nothing else counts as a gate.

Gate	Required attendees	What it validates
Gate 1	Domain authority + product owner	The Phase 2A set reviewed *together*: prototype, as-is flows, tentative ERD, sitemap, to-be sketches. A screen can be accepted while its data model is rejected, and only a joint review catches that.
Gate 2	DA + PO + at least one developer	All artifacts, model first and screens last: refined requirements, ERD, breakdown, to-be pairs, flow optimization deltas, IA, userflows, sitemap. Developer acceptance of the ERD is an exit criterion.
Gate 3	Internal team	Line by line: each in-list feature's spec against its REQ-IDs, plus an independent re-run of the status-mapping-versus-ERD reconciliation. Passing unlocks scheduling client validation, not shipping.

Reading a gate's outcome

> **THE QUIET GATE**
>
> Heavy correction at Gate 1 is **success**: the instrument worked. Little correction is a **warning**, not a pass. Either the artifact set was too safe to test anything, or the reviewer is being polite. Before a quiet gate may pass, the reviewer is probed ("complete the job on this build in front of me and narrate where it lies", "run month-end on this flow", "what does this let you do that reality forbids?"), and the report records that the probes were run.

Gate integrity rules

- **Attendance before agenda.** A missing required attendee means NOT HELD, recorded as such, rescheduled. No authority overrides this.
- **Readiness pre-checks.** The pre-gate critique has run with zero open blockers (or they are declared), the conformance snapshot is current, the assumption register is prepared to be read aloud. A gate entered on unknown state reviews artifacts that may already disagree with each other.
- **Disclosure before review.** Open assumptions and unresolved contradictions are read aloud *before* the review starts. Acceptance is only informed acceptance if they were stated first.
- **Written exit.** A verbal yes is recorded as `(asserted)` and the gate stays PASSED-PENDING-WRITTEN until confirmation arrives.
- **Post-gate verification within the week.** Every routed correction is checked to have actually landed. Corrections that evaporate teach everyone that gates are theatre.

2.5 Corrections: Type A, Type B, and the rework machinery

Every correction, from any source (a gate, a design critique, client validation, a refuted assumption, a conformance finding) is classified at the moment it arrives, in the room where possible.

Type	Changes	Handling
Type A surface	A screen, a flow drawing, wording, layout	The owning agent patches in place; a note goes in `STATE.md`; work continues.
Type B structural	A rule, an entity, a status, a permission, or a requirement	Routes to requirements-reconciler *first*, before any artifact is touched. The requirement entry is updated with a rationale, the version is bumped, dependency links are walked to mark every dependent stale, owning agents re-derive, and a rework-log row is written with the Phase 1 topic it traces to.

> **WHEN UNSURE, IT IS TYPE B**
>
> The cost asymmetry decides. A Type A misclassified as Type B costs one unnecessary trip through the reconciler. A Type B misclassified as Type A means the requirement document and the shipped screen now disagree, permanently, and nobody knows.

There is a diagnostic built on top of this. **Three or more Type B corrections tracing to the same Phase 1 topic** means discovery converged too early on that topic. The system escalates with a recommendation to re-open discovery on *that topic specifically*, not the whole phase.

```
# any correction, from anywhere
/rework "the DA says posted invoices can be amended within the same period"

→ classified Type B (touches a lifecycle rule)
→ routed to requirements-reconciler: entry updated, rationale written, version bumped
→ dependency links walked: erd.md, process-breakdown.md, 3 hi-fi specs marked STALE
→ staleness sweep run; rework-log row written
→ regeneration order reported, with the owning agent for each
```

2.6 Continuous conformance

Gates verify conformance at checkpoints, with humans in the room, at high cost. Between gates, the conformance engine verifies it continuously and cheaply, so gates spend their expensive minutes on domain judgment instead of on discovering that two files disagree.

The instrument is a **requirements traceability matrix**: every REQ-ID mapped to every artifact that implements, constrains or renders it. It is an *index of citations that already exist* in the artifacts, not a new bureaucracy; if a link has to be hand-invented to fill a cell, the finding is the uncited artifact.

Column	Content
REQ-ID	From refined-requirements.md, the single source of IDs
To-be flow	Which flow step(s) implement it, or NONE
Breakdown	Which rule row(s) enforce it, or NONE
ERD	Which entity or constraint carries it, or N/A
Hi-fi	Which feature spec(s) render it, or NOT-YET / DESCOPED
Verification method	inspection / demonstration / analysis / DA confirmation
Flags	ORPHAN, GHOST, STALE, CONFLICT

Two flags carry the weight:

- **ORPHAN**: a requirement nothing implements. Either it awaits its phase (fine, marked NOT-YET), it was descoped (and must appear on the descope list; if it does not, someone dropped it silently), or it was forgotten.
- **GHOST**: artifact behaviour citing no requirement. Either a missing requirement to route to the reconciler, or invention to remove. Ghosts are how scope creeps in quietly and how untraceable behaviour reaches production.

The four sweeps

Sweep	What it checks
Traceability	Rebuilds the matrix from current citations; diffs against the previous one. New orphans, new ghosts, links that vanished.
Staleness	Walks the dependency order. Did anything an artifact depends on change after its last edit? Mechanical, not judgmental: a stale artifact may still be correct, but nobody knows until its owner re-derives or re-affirms it.
Provenance	Untagged claims, (asserted) sitting where (confirmed) is required, [LLM] content outside its quarantine zones, assumption-register integrity and overdue decisions.
Drift	Direct document-versus-artifact comparison per layer: supersessions explicit, orphan checks both ways, ERD-versus-breakdown state machine in both directions, status mapping versus ERD, copy versus glossary.

Cadence

Trigger	Scope
End of each phase	Full snapshot
Before each gate	Full snapshot; it is a gate readiness pre-check
After every Type B lands	Staleness sweep plus affected matrix rows
Weekly during Phases 2B and 3	Traceability plus staleness
On demand: `/verify`	Caller's choice; default full

> **TWO DISCIPLINES THAT PROTECT THE MECHANISM**
>
> The evaluator **reports and routes but never fixes**: an evaluator that edits the artifacts it checks has no independence left. And **severity honesty**: BLOCKING is reserved for what genuinely blocks a gate, because inflating severity burns the mechanism's credibility. A snapshot older than its cadence is itself a finding.

2.7 Working without a spec

The most common real starting condition. Projects arrive with a slide deck, a two-page gap note, a Jira export, a login to an existing system, or a person saying "we will send it next week". The system makes that normal instead of blocking.

The spec slot

`00-inputs/PRD.md` exists in every project from the moment `/new-project` runs. It is one of exactly two things and never a third: the client's spec filed verbatim and read-only, or the completed placeholder record declaring the intake class and which substitute path is running. It is never absent, and never left as an unedited template. An unedited placeholder past Phase 0 is a process defect: it means nobody decided how this project would learn what it is building.

Six intake classes

Class	What arrived	Substitute path
A	A full spec	None. Audit it, then anchor-analysis normally
B	A partial spec	Audit marks thin areas; question generation weights them
C	A gap document	As-is reconstruction; gaps are problems, never requirements
D	Deck, notes, Jira export	Consolidate into a derived brief, then get it signed
E	An existing product	Product audit as the anchor; the build shows what it DOES, never what it should
F	Nothing	Reconstruction interview producing a signed discovery brief

Four or more ABSENT scores on the fourteen-dimension audit reclassifies a project to class D regardless of how professional the document looks. A bad spec is more dangerous than no spec, because it is trusted.

The reconstruction interview

You cannot ask "what do you want built"; people answer with solutions shaped by whatever software they last used. You ask people to narrate their world, and derive the spec from the narration. Six timeboxed passes: **goal** (what outcome, measured how), **actors** (who touches it, how often, who overrides whom), **artifacts** (what records move through it, seen as real documents where possible), **rules** (what must always be true, and which are legal rather than preference), **exceptions** (when it breaks, who fixes it, at what cost), **consequences** (if this were wrong for a month, what damage, discovered by whom, when).

The output is a derived brief, structured like a spec so the same fourteen-dimension audit can score it. Signed, it becomes the anchor document. Unsigned, it caps everything downstream at `(asserted)`, and every gate is told so in its first sentence.

The five-rung ladder

Any agent hitting a missing answer follows this, stops at the first rung that produces something honest, and marks which rung it used:

Rung	Source	Rule
1	Knowledge base	Answered with a citation, or it says it cannot answer
2	Client evidence	Existing build, exports, real documents. `[DOC:source] (asserted)`
3	Domain authority	Queue it; escalate DA-ONLY if it blocks today
4	Research	Informs a design decision; never sets a client rule
5	Assumption register	With blast radius, cheapest test, named owner, decision date

> **RUNG 5 IS LEGITIMATE. SILENCE IS NOT.**
>
> What is never allowed is skipping silently to rung 5, or producing an artifact whose reader cannot tell which parts rest on rung 1 and which on rung 5. Every artifact built on assumptions carries an ASSUMPTIONS box near its top listing the register IDs. Assumptions constraining behaviour may not pass Gate 2. A refuted assumption is a Type B correction by default.

2.8 The five adaptation variables

Read from `PROJECT_CONFIG.md`. They change routing; they are never guessed. If unset, the orchestrator asks and records the answers before Phase 1 work starts.

Variable	Values	Routing effect
`anchor_document`	supplied / none / existing-product	*none*: a signed discovery brief must be produced first. *existing-product*: as-is work becomes an audit of the build; Phase 1 weights toward audit.
`domain_authority`	sme / end-users / both	*end-users* or *both*: adds user interviews to Phase 1 and usability testing to Phase 3. *sme* only on a user-facing product logs risk R-SME-SUB.
`rule_density`	high / low	*low*: ERD modelling and process breakdown become optional, and the time reallocates to UI exploration. *high*: both are mandatory and developer-reviewed at Gate 2.
`engagement_type`	fixed-bid / t-and-m / augmentation	*fixed-bid*: the prioritized list locks at Gate 2 and later changes are scope changes requiring escalation. *augmentation*: client documents may replace artifacts, recorded as substitutions.
`designer_engagement`	demo / in-team / ready-to-code / modernize	*demo*: the prototype IS the deliverable, so throwaway framing and the archive rule are disabled and the quality bar rises. *ready-to-code*: the full shared-specs set is mandatory at handoff.

Two more the design layer needs, asked at project start because nobody volunteers them: **accessibility obligation** (sector, procurement standard, or contractual commitment above the AA floor) and **device and environment per role** (what people use, where, and with what on their hands). Unknown on either becomes an assumption register entry with a large blast radius; unasked, the second defaults silently to desktop, which is the most commonly late-discovered constraint in this process.

`minimum_set: true` reduces the mandatory artifact set for small or fast projects to: attributed problem statements, knowledge base, as-is flows, refined requirements, to-be flows, prioritized feature list, and hi-fi for prioritized features. Everything else becomes optional, but its skill still applies when invoked.

2.9 The design floor

These are constraints, not preferences. They override any conflicting instruction in a style guide, a client brand guideline, or a stakeholder's stated preference. Where a conflict is real, it is escalated, never resolved by lowering the floor.

NON-NEGOTIABLE

- **WCAG 2.2 level AA** on everything designed. Higher where a client's sector or procurement standard requires it; establish which at project start.
- **Meaning is never carried by a single channel.** Not colour alone, not position alone, not motion alone. The greyscale test settles it.
- **Every task is completable by keyboard alone**, with focus always visible and never removed without an equally visible replacement.
- **Interactive targets are never below 24 x 24 CSS px**, and 44 x 44 on touch-primary surfaces, regardless of density mode.
- **Nothing flashes more than three times per second**, and `prefers-reduced-motion` is honoured.
- **Every component is specified in every state it can reach**, including every ERD status value it can display. A component specified only in its default state is not specified.
- **Status values come from the ERD's closed lists.** A status appearing in a design that the ERD does not contain is a Type B correction, never a design choice.
- **No affordance the lifecycle forbids.** A delete control on a reversed-not-deleted entity does not exist anywhere in the interface.
- **Every style guide rule is violation-checkable.** If a reviewer could not point at a build and cite it, it is an aspiration and does not ship.
- **Real copy in every state before Gate 3.** Placeholders are incomplete specs.
- **An accessibility claim states its method.** An automated scan is never reported as an accessibility assessment.

PART 3

The phases, step by step

What actually happens in each phase: the sequence, who owns what, what is produced, and what has to be true before the next thing starts. Read the phase you are in; skim the rest.

3.1 Phase 0: Intake PHASE 0

Owner: discovery-lead · **Command:** `/new-project` then `/intake-spec`

A real phase, not a formality, and the only step that is never skipped, including when a spec is promised for next week. It decides what this project actually received and therefore how it will learn what it is building.

Sequence

1. **Inventory** every file, link, login and named conversation offered as input: what it is, who gave it, date.
2. **Classify** the set (A through F) and record it in `PROJECT_CONFIG.md` and `STATE.md`.
3. **File and freeze** everything into `00-inputs/`, read-only, with origin-and-date filenames. Client documents are never edited; corrections happen downstream.
4. **Fill the header block** on `00-inputs/PRD.md`.
5. **Audit** against the fourteen dimensions, scored PRESENT / PARTIAL / ABSENT. Resist rounding up: the audit's entire value is in its ABSENTs.
6. **Convert absences** into questions (grouped by who can answer) or register assumptions (where nobody can).
7. **Open the assumption register** with blast radius, cheapest test, named owner and decision date per entry.
8. **Start the substitute path** for classes C through F.
9. **Disclose** a missing spec in `STATE.md > blockers` so it appears in every gate report until a signed brief exists.

The fourteen audit dimensions

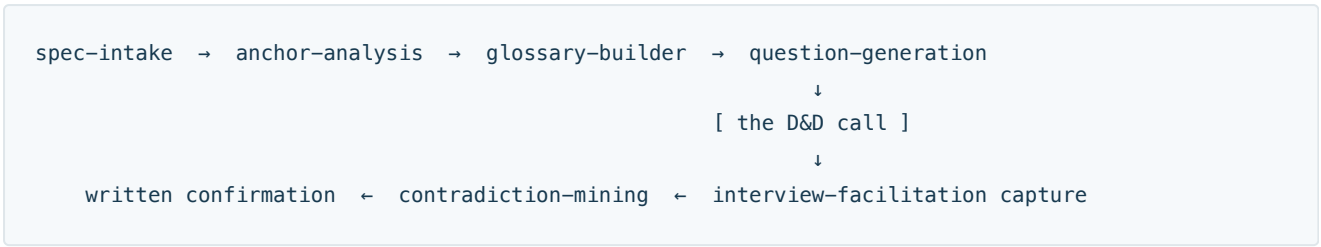
1. Business goal and measure
2. Users and roles
3. Entities and vocabulary
4. Rules and constraints
5. States and lifecycle
6. Permissions
7. The current process
8. Exceptions and failure handling
9. Integrations and data boundaries
10. Non-functional expectations
11. Platform and environment
12. Success and failure scenarios
13. Out of scope
14. The client's own open questions

Dimension 14 is the strongest single quality signal a spec can carry: a document admitting no unknowns was written to persuade, not to specify. Dimension 2 is the highest-leverage absence, because role count drives artifact count for the whole project.

3.2 Phase 1: Empathize and Define PHASE 1

Owner: discovery-lead · **Mode:** divergent, then convergent

Phase 1 is one activity, the discovery and definition call, and everything before and after it serves that call. The phase exits on a problem definition the domain authority has confirmed *in writing*.



Before the call

Anchor analysis reads the document forensically in three passes: as the intended reader (absorb what it wants you to conclude), as an extractor (the full claim typology, every item tagged `[DOC:section]`), and as a skeptic (what would a skeptical reader conclude from the same facts?). Two outputs matter most. The **commitment scan** captures every "must / shall / always / never / required by" sentence with its authority class (law, contract, policy, or preference) and its blast radius; the authority class is what later stops prioritization treating a 2014 policy choice as untouchable law. The **silence pass** asks eight questions of every described process (before, after, failure, volume, exception, authority, boundary, reversal) and records every unanswered one as a finding.

Glossary building classifies each term descriptive or rule-bearing using one test: *if two people defined it differently, would the product behave differently?* Rule-bearing terms get a dependency count, verification probes, and a place on the DA-verification list sorted by how many artifacts will rest on them.

Question generation derives the set from the analysis: one process question and one rules question per process, exception questions from the silences, an edge-testing question per commitment, an authority question per commitment lacking one, and a verification question per single-mention constant. Every primary question gets a 2 to 4 rung ladder written in advance (concretize, edge, actor, stake), and every question passes a lint: no yes/no primaries, no embedded answers, no hypotheticals as primaries, one question per question. Twelve well-laddered questions beat thirty flat ones; if the list exceeds the slot, cut whole questions rather than compressing all of them.

During the call

Three-column capture, `[PO]` `[DA]` `[TEAM]`, plus side lists for solutions offered, new terms, and requirement unfolding (every question a team member asks, logged with whether it was answered). Separation at the moment of capture is the whole point: separation after the fact is reconstruction.

Technique matters more than the script. Three to five seconds of silence after an answer, then the echo probe (repeat their last few words with a question tone) before reaching for the next written rung. Every abstract answer gets "walk me through the last actual time" before the conversation moves on. Four tells are worth probing when they appear: a hesitation (uncertainty or politics), a laugh (usually a workaround), the word "just" (compression, ladder down), and "obviously" (an assumption the client cannot see).

When the conversation and the document disagree, say so out loud, neutrally, and capture both. That disagreement is among the most valuable findings the call produces. When the DA and PO debate each other, let it run while it stays concrete and capture both sides tagged; live disagreement is high-grade evidence.

The closing summary is never cut for time. Summarize back group by group, then ask verbatim: what did I get wrong, what did I not ask about, who sees this differently, what are you worried we will get wrong. Mishearings caught here cost a sentence; caught at Gate 2 they cost a re-model.

After the call

Contradiction mining runs in two stages. A model-assisted sweep with the exact framing "list every contradiction between these sources; do not resolve, reconcile or merge anything", then a human filter applying the co-satisfiability test: *if both statements were true, could the same product satisfy both?* Yes means phrasing, drop it.

No means real, keep it. Then a deliberate under-detection pass, because models over-detect surface phrasing and under-detect deep conflicts.

Problem statements carry five fields: problem, source tag, current workaround, downstream consequence, confidence. A problem with no stated workaround queues the workaround question (people always cope somehow; an unknown coping mechanism means the problem is not yet understood). A problem with no downstream consequence is suspect and queues the stake question.

> **EXIT CRITERIA, ENFORCED**
>
> Written confirmation of the problem set from the DA (a verbal yes upgrades nothing), zero owner-less contradictions, and `open-questions.md` converted into the Phase 2A research plan. The phase can close on an explicit, recorded risk acceptance, but never on silence.

3.3 Phase 2A: Initial Design PHASE 2A

Mode: divergent, parallel workstreams

Six workstreams, with real dependencies between them:

```
domain-researcher ─┐
kb-curator ────────┼─→ prompt-spec-writer ─→ prototype-engineer
flow-modeler ──────┘         (needs kb-index)      (needs a complete prompt)
  (as-is)

ui-designer  ── density decision + status encoding sketch ONLY
data-modeler ── tentative ERD sketch
flow-modeler ── tentative structure set
```

Research, knowledge base and as-is mapping start immediately and in parallel. Prompt writing needs the KB index to exist. Prototype generation needs a complete prompt: no prompt, no prototype, no exceptions.

The ui-designer deliberately does *not* build a system here. In 2A they settle the density decision and a status encoding sketch and nothing else, because a full visual system authored against an uncorrected entity model is half thrown away at Gate 1.

Controlled roughness

The tentative set (sitemap, to-be sketches, userflow sketches, ERD sketch) exists so Gate 1 has structure to reject, not just a prototype to react to. An authority who accepts a screen will often reject the data model behind it once both are visible. Three disciplines keep it useful:

- **Timeboxed**, declared in `STATE.md`. Roughness is enforced by the clock. If a diagram is taking long enough to look finished, it has exceeded its purpose.
- **Derived, not invented.** Every element carries a sources line ("derived from: as-is/invoicing rows 4-11, problem P-07"). Free-floating structure is decoration, and it survives unexamined precisely because nobody owns the claim behind it.
- **Uncertainty marked on the artifact**: `(inferred)` labels, "?" edges, and an open-questions box in the corner of each sketch listing the two or three things the team most wants ruled on. Reviewers correct what is visibly uncertain and politely ignore what looks decided.

The prototype's fidelity asymmetry

Build properly	Leave obviously rough
Realistic content at realistic volume, including one negative, one zero and the longest plausible name	Visual polish: colour, typography, shadows, icon craft
Every status the sketch entities can hold, visible somewhere	Anything the scope boundary excluded
The empty state and at least one error state	Animation and transitions
The complete happy path of the job, end to end	Auth, settings, exports
Native, keyboard-operable controls with visible focus	Responsive behaviour, unless a role genuinely works on a tablet
The table at the client's real row count	Perfect alignment and off-scale nudging

The reason is asymmetric. A domain authority looking at five status badges will name the sixth, which is a Type B correction worth days. A domain authority looking at slightly wrong spacing will comment on the spacing, which is worth nothing at this gate. Visible roughness in the visual layer actively protects the gate's attention.

3.4 Gate 1 GATE

Attendees: domain authority + product owner · **Command:** `/gate-1`

The set is reviewed *together*, never artifact by artifact. The prototype is introduced with its framing script out loud: disposable, built to be wrong in a useful way, tell me where it is wrong. Then the six questions, in order:

1. Where is the as-is process factually wrong or incomplete?
2. Which steps are missing?
3. Which entities or relationships are wrong?
4. Which edge cases would break this?
5. Which terminology is wrong for this domain?
6. What must be true that is not shown here?

The second half of the agenda is the tentative set's open-questions boxes: the team asked for specific rulings, and this is where they get ruled.

On pass: corrections classified and routed, the prototype and its prompt archived to `50-prototype/ARCHIVED/`, the report filed, phase set to 2B, and reconciliation announced as the next action.

3.5 Phase 2B: Further Design PHASE 2B

Mode: convergent. **The chain is strictly ordered and the order is enforced.**

```
data-modeler          permission matrix, closed status lists, lifecycle, consequences note
      ↓
flow-modeler          to-be flows that are CORRECT, process breakdown with its cross-checks
      ↓
ux-architect          optimize (cost tables) → structure (IA) → draw (userflows, sitemap)
      ↓
ui-designer           the visual and interaction system
      ↓
hifi-spec-designer    Phase 3 screens
      ↓
design-critic         before each gate
```

Each arrow is a seam with a reason:

- Optimizing before the process is correct optimizes the wrong process.
- Drawing userflows before the structure is chosen means redrawing them.
- Building screens before the system exists produces one-off exceptions that fork the system.

Requirements reconciliation runs first and continuously

The refined requirement document is seeded from anchor-analysis commitments, problem statements, the PRD draft, promoted register assumptions, and every Gate 1 Type B correction. Every Resolved statement passes five quality rules: **testable** (a reader can construct the passing and failing case), **atomic** (an "and" joining two obligations is two entries), **solution-free** (what must be true, not which widget), **vocabulary-bound** (every term in the glossary), and **quantified where quantifiable**.

Dependency links (depends-on, feeds) are wired per entry. They are what turn Type B impact analysis into a lookup instead of an archaeology dig.

The ERD is where the expensive mistakes live

The exhaustion protocol: collect candidate status values from every source (flows, the document, incumbent filter dropdowns, the current build's badges), read the list to the DA, and ask "and what else can it be?" until the answer is nothing. Then probe the gaps: what is it between X and Y, what is it while it waits, what if the user abandons it, has one ever been in two at once (if yes, there are two status fields wearing one name). Finish with the terminal check.

Cardinality by counterexample, never by rule: "has there ever been an X with two Ys?" The dangerous answers are "normally one" (model n, display for one) and "it depends" (chase the condition; it is a rule).

The temporal pass: per attribute referencing another entity's data, is the value frozen at-the-time or a live current lookup? The probe is "if the price changes tomorrow, should last month's invoice change with it?" The answer is almost always no, and getting this wrong corrupts history silently.

The consequences note is the deliverable that carries all of this to the designers, one line per rule that constrains the interface: "X is reversed, never deleted: no delete affordance anywhere; provide reverse with a reason field."

To-be design and the improvement ladder

Applied in order to every pain-marked as-is row:

Rung	Meaning
1. Eliminate	Remove the need. Ask why the step exists before how to speed it up
2. Simplify	Fewer decisions, inputs, touches; same outcome
3. Integrate	Kill the re-keying and the handoff seam; data moves itself
4. Automate	Last deliberately: automating a step that should have been eliminated fossilizes it

Before any elimination, the **second-job check**: a redundant-looking step often does control, audit trail, training or handoff signalling invisibly. Ask "who would notice if this stopped happening, and when?"

Automation honesty: a step is automatic only when its trigger and inputs are fully specified by `(confirmed)` entries, and every automatic step specifies detection, silence window, fallback and override. Automation does not remove the unhappy path, it relocates it.

Flow optimization: counting before proposing

Every task gets a cost table: decisions, inputs classed K (known to the system), R (retrievable) or N (genuinely new), waits, navigations, recalls, with the as-is totals beside them. Every K and R input is a defect until proven otherwise: the system is asking a person for something it already has. A to-be that does not reduce the totals is a re-platforming, and that is worth knowing before Gate 2 rather than after launch.

Friction class	Default fix
Unnecessary decision	Default it, visibly and changeably
Unnecessary input	Prefill, derive, or look up. Show it, do not ask for it
Unnecessary wait	Move it off the critical path
Unnecessary navigation	Bring it to the point of decision
Unnecessary recall	Show it at the moment it is needed
Unnecessary rigour	Downgrade per the friction budget

The last class is the one designers under-apply. Friction added out of caution costs the user on every single pass and pays off only on the rare error, and nobody audits it because it looks responsible. The friction budget sizes ceremony against the consequence data from discovery: detection latency, remediation cost, reversibility.

Information architecture

Structure derives from the entity model plus roles, never from the feature list; the feature list is a coverage check run last. Labels come from the glossary, always: a label with no glossary entry is a defect in one of the two files. The navigation model is chosen against counted inputs (primary entities, roles, role overlap, daily tasks per role, rare-but-critical tasks), and the rejected alternatives are recorded, because six months later someone will propose them again.

The structure is *validated, not asserted*: a tree test (eight to twelve real tasks, "where would you go to...?") and a reverse card sort, an hour with the DA and two or three users. Anything under roughly 70 percent direct success on a daily task gets reworked before it is drawn.

Prioritization closes the phase

The mandatory partition is separated first and audited against the authority classes captured back in anchor analysis: only law and contract are automatically mandatory; policy and preference compete like everything else. Only the optional partition is scored, with a method chosen and recorded (Impact/Effort, MoSCoW, RICE, WSJF, Kano, opportunity scoring, or P0 ranking), and every estimate carrying its provenance. Then the two domain rules apply, after scoring, visibly:

> **THE TWO PRIORITIZATION RULES**
>
> **1. Mandatory is not a candidate.** Anything a `(confirmed)` DA rule or a `[DOC]` commitment requires is in by definition. A method that scores a legal requirement low has mis-scoped its inputs, not discovered a saving.
>
> **2. Process completeness over feature count.** A half-supported process is worth less than no support: it forces users to run two systems at once. Rank in units of complete to-be flows, including their exception paths.

3.6 Gate 2 GATE

Attendees: DA + PO + at least one developer · **Command:** `/gate-2`

Review order is enforced: refined requirements, ERD, breakdown, to-be pairs with their pain-index deltas, flow optimization deltas, IA, userflows, sitemap. **Screens last.** Screen-first review hides model contradictions, reliably.

The developer's attendance is the point, not a courtesy. Their acceptance of the ERD and their read of the breakdown are exit criteria; their structural objections are cheap now and multiples later. Hand them the two-developer-test results as a reading guide.

On pass: the prioritized list is signed in writing (marked LOCKED on fixed-bid), the prototype is confirmed archived, and Phase 3 starts from the list and the consequences note.

3.7 Phase 3: Test and Prototype PHASE 3

Owner: hifi-spec-designer, with ui-designer and design-critic

Convergent and divergent at once: each prioritized feature converges to production intent, and anything discovered that changes a requirement diverges back through the reconciler.

The state enumeration table is built before any screen. Feature by entity by status, from the ERD's closed lists rather than from memory. Each cell is a screen state to specify or an explicit "not reachable here" proved with the flow. Unproven unreachability is a missing screen. Building this table first is what stops states being discovered late and expensively.

Nine blocks per feature spec

#	Block	Contents
1	Screen states	The generic set (empty, loading, partial, error, success, permission denied) plus one state per ERD status value the feature's entities can hold
2	Screen composition	The three hierarchy levels named in writing before drawing; layout, density, grouping
3	Dev notes	Behaviour the visual cannot carry: validation rules with REQ-IDs, calculation logic, what recalculates when, server-authoritative versus client
4	Interaction guides	Feedback timing, optimistic/pending/confirmed per mutation, validation timing, save model, keyboard contract, focus movement, concurrency
5	Status mapping	Per status: trigger, who may trigger, which transitions are irreversible, what every role sees
6	Screen flow	How screens connect, exception paths drawn rather than implied
7	Real content	Actual copy in every state, REQ-IDs beside any rule-bearing string
8	Accessibility contract	Element or ARIA pattern, name source, keyboard pattern, states exposed, announcements, target sizes
9	Responsive adaptation	Breakpoint behaviour, dense-data strategy, target sizes by environment

Partial specs are marked DRAFT in the header. No silent partials, and no DRAFT headers left at gate time.

> **PROTOTYPE GRAVITY**
>
> Sources are the refined requirements, the ERD, the flows and the IA. **Never the archived prototype.** If you find yourself opening `ARCHIVED/`, stop and say why out loud. Hi-fi that quietly refines archived screens inherits every uncorrected assumption in them.

3.8 Gate 3 GATE

Attendees: internal team · **Command:** `/gate-3`

Line by line: each in-list feature's spec against its REQ-IDs. Then an *independent* re-run of the status-mapping-versus-ERD reconciliation, separate from the spec designer's own self-check, because that mismatch is the most common Gate 3 defect. Critique lenses 3, 4 and 7 (state completeness, accessibility, domain fidelity) are also re-run independently, because those three are where a self-critique is least reliable: they require reading the ERD rather than looking at the screen.

Passing Gate 3 unlocks *scheduling client validation*, not shipping.

3.9 Handoff and the md-spec loop CLOSE

Owner: handoff-manager

Handoff is a transfer of understanding, not a file drop. The measurable goal: build-time clarification requests approach zero for anything the artifact set already answers.

The what-if sweep, before the walkthrough

Per in-list feature, confirm that an *artifact* (not your memory) answers each of: every mutation's optimistic/pending/confirmed decision and failure behaviour; every screen state including ERD statuses; every exception path's landing state and resume point; every irreversible transition's confirmation pattern; concurrency per multi-user surface; the narrow strategy per dense table. Anything unanswered is patched that week or listed as a manifest gap. The sweep list itself is filed so the dev team can see what was checked.

The walkthrough

Live, developer present, in manifest order: requirements, ERD with the consequences note read aloud, breakdown, flow pairs, userflows and sitemap, then per-feature specs. Model before screens, same logic as Gate 2.

Two steps are easy to skip and worth protecting. **Read the glossary's rule-bearing terms aloud with their definitions**: the dev team will name variables and tables after these words, and those ten minutes are where the ubiquitous language either transfers or does not. **Walk one feature end to end at full depth** before surveying the rest: it calibrates how much answer the artifacts carry, which is what stops the team asking a person instead of reading.

Then land the routing rule explicitly: build-time questions about rules go through the reconciler path, not a designer's memory; questions about visuals go to the style guide and skills-md first. Show them the rework log so they see the mechanism working.

> **THE HANDOFF'S OWN QUALITY SCORE**
>
> Split the question list into **artifact-answered** and **memory-answered**. Every memory-answered question is a spec gap to patch that week, not someday. A walkthrough that went well because the designer answered everything live is a walkthrough whose answers leave when the designer rolls off.

The md-spec loop

With a developer, never solo. Gather the evidence first: this project's style guide and skills-md deltas with their recorded reasons, the dev team's actual built conventions read from real code, and the build-time question log. Then adjudicate every divergence with one of three outcomes:

Finding	Outcome
Diverged deliberately (reason recorded)	Adopt into the shared file, or keep project-local and say why
Diverged silently	Adopt or revert, but record either way; silent forks are the decay mechanism
Shared file asserts something nobody follows	Delete it. Aspirational rules rot trust in the whole file

The adopt-versus-local test: would the rule hold on a different client with a different brand? Yes means shared. Consider at least one deletion honestly at every review; a file that only grows becomes a file nobody reads.

The three loop metrics

Recorded per project and compared across projects. All three should fall as the shared files mature. They are cheap (each is a count) and they are the difference between "we think the process works" and evidence.

Metric	A rise means
Requirements changed after Gate 2 (post-G2 Type B count)	Discovery or reconciliation is converging too early
Build-time clarification requests	The breakdown and spec layer is under-specifying; find the cluster
Share of generated front end re-used rather than rewritten	The shared-specs files or the prompts are drifting from real conventions

PART 4

The agents

Fourteen specialists. You rarely address them directly: describe what you need and the orchestrator routes it. Each card below gives the role, the skills it loads, what it receives and hands over, the bar it holds itself to, and when it stops and escalates.

4.1 How agents, skills and the orchestrator relate

```
ORCHESTRATOR (CLAUDE.md)   sequences phases, enforces gates, maintains STATE,
                           routes work. Does no specialist work itself.
        │
        ├── AGENT (.claude/agents/*.md)   a role with duties, handoff contracts,
        │      │                          a quality bar, and escalation triggers
        │      │
        │      └── SKILL (.claude/skills/*/SKILL.md)   the craft: procedures, decision
        │             │                                 tables, probing banks, failure modes
        │             └── references/*.md               depth loaded on demand
        │
        └── GUARDRAILS.md   loaded by everyone; overrides everything on conflict
```

The division is deliberate. Skills carry depth so that craft improves in one place. Agents are thin routers with role-specific duties and, critically, **handoff contracts**: who they receive from and who they hand to. The orchestrator only sequences. A change to how ERDs are modelled happens in one skill file and every agent that loads it improves.

4.2 The routing table

Work	Agent
Spec intake, PRD placeholder, no-spec reconstruction, assumption register	discovery-lead
Anchor doc analysis, glossary, questions, running and consolidating the D&D call	discovery-lead
Concept research, incumbent product study and teardowns	domain-researcher
Building, structuring and querying the knowledge base	kb-curator
PRD draft, prompt authoring, skills-md structure, content and microcopy rules	prompt-spec-writer
As-is flows, to-be flows, process breakdown, tentative structure	flow-modeler
ERD, status fields, permissions, record lifecycle	data-modeler
Information architecture, flow optimization, userflows, sitemap	ux-architect
Style guide, tokens, layout, type, colour, components, states, motion, responsive	ui-designer
Generating, timeboxing and archiving the throwaway prototype	prototype-engineer
Refined requirement document, correction reconciliation, prioritization	requirements-reconciler
Hi-fi specs, screen states, status mapping, dummy content	hifi-spec-designer
Pre-gate critique, accessibility, content quality	design-critic
Any gate, any review, correction classification, conformance snapshots	gate-reviewer
Handoff package, walkthrough, md-spec loop	handoff-manager

4.3 Agent cards

discovery-lead PHASE 0 PHASE 1

Skills: spec-intake · anchor-analysis · glossary-builder · question-generation · interview-facilitation · contradiction-mining

Goal. Leave Phase 1 with a problem definition the domain authority has confirmed in writing, a clear separation between what is known, assumed and open, and every contradiction surfaced with an owner. Everything downstream inherits the quality of that separation.

Receives: whatever the client supplied, or nothing. **Hands over:** the intake class and audit to everyone; the process inventory and actor list to flow-modeler; term lists to the glossary and to data-modeler for entity seeding; solution hypotheses to requirements-reconciler (reconciled in 2B, never earlier); open questions to domain-researcher and kb-curator; the confirmed problem set to requirements-reconciler.

Quality bar. Nothing untagged. At least one silence per described process. Every commitment covered by a question or waived with a reason. Every capture line attributable without guessing. Every kept contradiction with both quotes, both tags, a type, a rank, an owner and a due date. Zero `(inferred)` entries in the problem set.

Escalates when: no domain authority is identified; Phase 0 is being skipped; the PO and DA disagree on the core problem and neither will own it; the call happened without recording consent; two chases have produced no written confirmation.

domain-researcher PHASE 2A

Skills: incumbent-research · **Tools:** adds WebSearch and WebFetch

Goal. Give the team the domain's accumulated knowledge without letting any of it harden into a client rule. Everything produced informs a design decision; none of it decides one.

Method worth knowing. Reads the data model out of the UI: filter values are the status enum, columns are the attributes, what an edit screen refuses to change after a point reveals the lifecycle. Runs task walks on the project's top three tasks, scores heuristic findings 0 to 4, and records every *refusal* (what a product will not let users do), because refusals encode hard-won rules.

Hands over: implied ERD sketches and observed status lists to data-modeler; mental models and convention findings to ux-architect; density evidence and harvested edge-case states to ui-designer; completed batches to kb-curator.

Quality bar. Notes column-comparable across products. Every load-bearing claim graded (A primary / B practitioner / C marketing / D unattributed) with an access date. Nothing in the output reads as a client rule.

Escalates when: only the DA can answer (tag DA-ONLY and stop researching); sources conflict on something load-bearing (record both, do not pick).

kb-curator PHASE 2A 2B

Skills: knowledge-base

Goal. Make it possible, at any point during the project and after it, to answer "where did this come from?" about any claim in any artifact. Also the mechanism that keeps a model-augmented team honest: an agent that must answer "per which file?" cannot drift into fluent invention.

Method worth knowing. Three groups never mixed in one file (client, authority, external), and they *rank*: authority beats client beats external. The **upgrade scan** runs on every authority ingestion, looking for statements that upgrade existing `(asserted)` entries elsewhere and notifying their owners. That scan is how written confirmations actually propagate instead of sitting unread in a folder.

Quality bar. Zero unindexed files. Every authority ingestion ran its upgrade scan. Conflict marks current. Three random index rows still match their contents.

Escalates when: an agent repeatedly answers from memory what the KB could answer; client material is about to be mixed into a research file or sent externally; a `(confirmed)` requirement cites an authority item that does not exist or does not say it.

prompt-spec-writer PHASE 2A

Skills: prompt-authoring · style-guide-authoring · content-design

Goal. A prompt is a design document: the artifact you keep, iterate and archive with the prototype. Refining a prompt costs minutes; regenerating a prototype costs hours, and this agent owns that trade.

Method worth knowing. Six mandatory parts (context, job to be done, data model sketch, constraints, style reference, scope boundary) plus a **carry checklist** that lives inside those parts: every status the entities can hold, the named density mode, the realistic-content instruction with volume, an explicit request for empty/loading/error states, the accessibility floor, and what is deliberately stubbed. Their absence is the most common cause of a build that looks right and teaches nothing.

Quality bar. All six parts plus every carry row. Zero untagged domain facts. The stranger test run honestly: name the two places a stranger would diverge, then tighten or accept them knowingly. Every skills-md file under about 150 lines with a precedence header.

Escalates when: a constraint would rest on `[LLM]`-only provenance; shared-specs are stale or missing on a ready-to-code engagement; part 3 is mostly "unknown", which means the prototype is premature.

flow-modeler PHASE 2A 2B

Skills: as-is-mapping · tentative-structure · to-be-design · process-breakdown

Goal. Prove the process is CORRECT. (ux-architect makes it GOOD.) These artifacts are the evidence base every later argument rests on: the pain index justifying the redesign, the rules that stop developers guessing, and the exception rows that decide half the screens in Phase 3.

Method worth knowing. Marks each step VISIBLE or BACKSTAGE from the requester's point of view and flags every crossing, because "they phone us to ask" is one of the most valuable findings the mapping produces. Computes a **pain index** per flow (manual data movements, uncaught-error rows, total wait, crossings without status communication, systems touched), which becomes the baseline the to-be must beat. Runs three cross-checks on the breakdown, including the state-machine check against the ERD *in both directions*.

Quality bar. No silently-inferred bridging steps. Both problem-reconciliation directions run. Every pain-marked row addressed with its ladder rung and second-job result. Every automatic step with all four failure-mode answers. Zero uncited rules; zero reason-less N/A exceptions. Two-developer test run per section.

Escalates when: only the DA knows a segment (tag DA-ONLY, never bridge by inference); a gate correction changes a rule inside a flow; an exception needs a status the ERD does not contain; a to-be does not beat its as-is pain index.

data-modeler 2A sketch PHASE 2B

Skills: erd-modeling

Goal. A model whose status lists are closed, whose lifecycle rules are confirmed, and whose consequences for the interface are written in language a designer can act on. Every status value missed becomes a missing screen in Phase 3; every implicit lifecycle rule becomes an affordance that legally cannot exist.

Method worth knowing. The split test applied both ways (two lifecycles wearing one name means split; two names never differing means a glossary collision). The exhaustion protocol for status lists. Cardinality by counterexample. The temporal pass separating frozen at-the-time copies from live current lookups. And the **consequences note**, which is how lifecycle rules actually reach the screens.

Quality bar. Every entity carded or explicitly collapsed. Every status list closed WITH the DA, not from memory, with the terminal check run. Every cardinality carrying its counterexample answer. Zero `(asserted)` lifecycle rules outside open items. The consequences note acknowledged by its three consumers.

Escalates when: the DA cannot rule on a lifecycle question; the developer raises a structural objection; retention and erasure obligations conflict; a status value is discovered after Gate 2.

ux-architect PHASE 2B 3

Skills: information-architecture · ux-flow-optimization · userflows-sitemap

Goal. The experience layer between the process model and the screens. flow-modeler proves the process is correct; this agent makes it good, gives it a structure, and renders it per role.

Method worth knowing. Sequence matters: optimize before you place (a step you delete needs no home), then decide the structure, then draw. Count before proposing: the cost table with as-is baselines. Apply the elimination ladder from the top and record which rung each fix landed on, because speeding a step that could have been deleted is the most expensive habit in this role. Size friction against consequence, not against caution.

Quality bar. Every prioritized task has a cost table with a stated delta. Every K or R input removed or proven necessary. Friction budget sourced or registered. Navigation model recorded with rejected alternatives. Every label traced to a glossary entry. Tree test run and sub-threshold paths reworked *before* drawing. Every denied branch designed per case.

Escalates when: consequence data for a friction budget does not exist; an optimization would change a mandatory rule; a daily task tests below roughly 70 percent and the fix would change the entity model; two roles need structures so different that the product is arguably two products.

ui-designer 2A seed 2B 3

Skills: visual-design · component-architecture · interaction-design · responsive-design · style-guide-authoring

Goal. The system every screen is built from: tokens, components, states, behaviour and adaptation. Judged by one test throughout: *could a reviewer point at a build and cite the rule it violates?* Anything failing that test is an aspiration and does not ship.

Method worth knowing. Domain decisions before aesthetic ones: density from the users and the volume, status encoding from the ERD's closed lists, target sizes from the environment table. Systematic and purposeful: every measurement from a scale, every colour from a role, and every visual difference meaning something. Contrast verified *while* designing, because a failed token found at review repaints everything downstream of it. No component specified in its default state alone.

Quality bar. Every foundation expressed as a citable rule. Every ERD status with exactly one treatment passing the greyscale test. Contrast verified on real backgrounds in both themes including hover. Complete state matrices. Accessibility contract per component. Every mutation classified. Reduced-motion equivalents specified. Zero hover-only paths; no target under 24 x 24.

Escalates when: a brand colour fails contrast and the client wants it as body text; device context is unknown; a design needs a status the ERD lacks or an affordance the lifecycle forbids; shared-specs design-system is missing on a ready-to-code engagement.

prototype-engineer PHASE 2A

Skills: throwaway-prototype · **Tools:** adds Bash

Goal. Maximise corrections per hour of Gate 1. The build succeeds when the DA says "that is wrong, and here is why" a dozen times; it fails when everyone nods.

Method worth knowing. No prompt, no prototype, and the carry list is checked as well as the six headings. Generate from the prompt as written: every manual patch is prompt debt the next generation loses, so defects are traced to a prompt part and the part is fixed. Spend fidelity asymmetrically (see 3.3). Timebox declared before starting; exceeding it is a stop signal, not an extension request, and feeling attached to the build is the named symptom of drift.

Quality bar. The job is completable end to end, verified by actually completing it rather than inspecting screens. Every guessed status visible somewhere. Realistic content, never lorem ipsum. build-notes tracing every delta to a prompt part.

Escalates when: anyone proposes polishing the prototype into hi-fi outside a demo engagement; the timebox is exceeded; the build violates a `(confirmed)` constraint (a prompt defect, not a build defect).

requirements-reconciler PHASE 2B ALL GATES

Skills: requirements-reconciliation · prioritization

Goal. One source of truth every other artifact derives from and cites, with IDs stable enough that traceability is a lookup rather than an archaeology dig. Also the single door every structural correction passes through; that door is what keeps the documents and the design in agreement for the whole project.

Method worth knowing. A reconciliation, not a rewrite: the document shows where the original and the authority disagreed and why the resolution won. "The DA said so" is a tag, not a rationale; the rationale names the tiebreaker. Draft from the KB, then *edit by hand*: the hand edit is where the rationale gaps become visible.

Quality bar. Every entry passing the five quality rules or carrying its asserted-open reason. Rationale wherever sources disagreed. Dependency links present, testable by impact-tracing a random Type B by links alone. Conflict scan run. Zero owner-less open items. On the list: method recorded, every score carrying provenance, mandatory partition audited, completeness pass including exception paths, signatures in writing.

Escalates when: three Type Bs trace to one Phase 1 topic; a late correction is structural (pause hi-fi on affected features); a fixed-bid list change is requested after Gate 2; effort estimates are being made with no engineering voice; an entry oscillates twice.

hifi-spec-designer PHASE 3

Skills: hifi-spec · dummy-content · visual-design · content-design · accessibility

Goal. Specs complete enough that a developer builds from them without asking anything the artifacts already answer. The measure is external and tracked across projects.

Method worth knowing. The state enumeration table before any screen. The ERD consequences note opened *before* designing, not after review. Consume the system rather than re-invent it: where something is missing, ask ui-designer for a *rule*, because one-off exceptions are how a design system forks. Ambiguity found mid-spec is upstream debt: route it, do not resolve it locally with a plausible choice.

Quality bar. Every cell specified or proved unreachable with a flow path. All nine blocks, no DRAFT headers at gate time. Every irreversible transition telling the user *before* the confirmation. Real copy everywhere, reviewed as a list. Boundary audit clean. Self-reconciliation against the ERD filed. Keyboard path written per task.

Escalates when: a feature cannot be specced because a rule is only `(asserted)`; the list changes mid-phase on fixed-bid; a design needs a status the ERD lacks; the system lacks a rule and ui-designer cannot supply it before the gate.

design-critic PRE-GATE

Skills: design-critique · accessibility · content-design

Goal. Find, before the client does, everything the team could have found itself, so expensive gate minutes go on domain correctness. The second job is the opposite of the first: to *protect* correct decisions that are merely unfamiliar. A critique run against written rules distinguishes "this violates our density rule" from "this is not how I would have done it", and only the first is a finding.

Method worth knowing. Seven lenses, run *one lens at a time across the whole set*, never screen by screen: every screen can pass individually while the set is incoherent, and only a single lens run across everything sees that. A recurring finding is a rule problem, reported as a proposed rule change rather than eleven defects.

Quality bar. Every finding citing the rule it violates, or evidence-based, or labelled a preference and dropped. Perceptual and accessibility tests actually run and recorded. Zero open blockers, or declared in the gate's first two minutes. Type B findings routed before any fix.

Escalates when: a blocker cannot be fixed before the gate; accessibility conflicts with a brand guideline or client instruction; three Type B findings trace to one Phase 1 topic; the same violation recurs after a rule change (the rule is unclear, not unfollowed).

gate-reviewer ALL GATES CONFORMANCE

Skills: gate-review · process-evaluation

Goal. Make gates mean something, and between gates keep the artifact set verifiably in agreement with the documents it derives from.

Method worth knowing. Attendance before agenda. Readiness pre-checks before scheduling. Register and contradictions read aloud before review, so acceptance is informed acceptance. Classification in the room, because late classification drifts toward Type A (patching is easier than reconciling). Post-gate verification within the week.

Two disciplines on the conformance side: reports and routes but never fixes (an evaluator that edits what it checks has no independence), and severity honesty (BLOCKING reserved for what genuinely blocks, because inflation burns the mechanism's credibility).

Escalates when: anyone proposes passing a gate without required attendance or written sign-off; three Type Bs trace to one Phase 1 topic; a gate would run on an unknown state.

handoff-manager CLOSE

Skills: handoff-package · md-spec-loop

Goal. A dev team that can build without you, and a shared-specs set that makes the next project's generated output closer to real team conventions than this one's was. Both are measured, not asserted.

Method worth knowing. The what-if sweep before the walkthrough. The full artifact set, not the designs: a Figma-only handoff is a named failure of this process. Gaps listed AS gaps. The question split as the handoff's own quality score. The loop review with a developer, never solo, reading real code rather than intentions.

Quality bar. Manifest complete-or-gapped honestly. Sweep run and memory-answered gaps patched. Routing rule demonstrated, not merely stated. Every divergence adjudicated with a recorded reason. All three metrics recorded with sources.

Escalates when: a developer is unavailable for the walkthrough or the loop review (both are rescheduled, not run without them); close is requested with unresolved Type B feedback; ready-to-code with an incomplete shared-specs set; the loop would adopt a rule the dev team does not actually follow.

PART 5

The skills

Thirty-four skills carry the craft. Each is a folder with a `SKILL.md` entry point and, where depth warrants, `references/` files loaded on demand. This part gives each skill's purpose, the mechanisms worth knowing, and its output.

5.1 Skill anatomy

Every skill file follows the same shape, which is also the shape to follow when writing a new one:

Section	Contains
Frontmatter	`name` and a `description` that states what it does AND when to use it (this is what triggers it)
Purpose	Why this exists and what failure it prevents
Inputs	Which artifacts it reads, and what it does when they are missing
Method	The craft: decision tables, protocols, taxonomies, numbers
Procedure	The ordered steps
Deciding factors	Judgment calls and how to make them
Probing prompts	Questions to ask the client, and to ask yourself
Quality bar / Exit criteria	The self-check before handing off
Guardrails	What this skill may never do
Output	Exact file paths produced
Failure modes	What going wrong looks like, so it is recognisable early

Keep `SKILL.md` under about 300 lines. Depth beyond that belongs in `references/`, because a skill file too long to read is a skill file that gets skimmed.

5.2 Intake and discovery PHASE 0 PHASE 1

spec-intake

Classifies whatever arrived (six classes), audits it against fourteen dimensions, fills the spec slot, opens the assumption register, and runs the substitute path when there is no spec. The one step never skipped.

References: `spec-audit-checklist.md` (dimension by dimension, with what each absence converts into and how to read the resulting scorecard); `reconstruction-interview.md` (the six passes, with the questions, in order, and the anti-patterns).

Output: `00-inputs/PRD.md`, `spec-intake.md`, `assumptions.md`, `audit/`, and `10-discovery/derived-brief.md` for classes C through F.

anchor-analysis

Forensic reading of the anchor document: three passes (intended reader, extractor, skeptic), a claim typology routing each extraction to its destination, the commitment scan with authority class and blast radius, the eight-class silence taxonomy, author-intent analysis, and the internal contradictions list.

Why it cannot be skipped: questions written without it are generic, and a generic question set is visible to the domain authority within five minutes.

Output: `10-discovery/anchor-analysis.md`, including the verification agenda ordered by blast radius, which is what question-generation prioritizes from.

glossary-builder

Builds the project's ubiquitous language: definitions with includes and excludes, descriptive versus rule-bearing classification, DA verification with recorded probes, client-versus-industry pairs, and collision policing across every artifact for the project's whole life.

The three collision patterns: two names for one thing (pick one with the DA, record the alias); one name for two things (split into qualified terms); and a faction split (a contradiction with an owner, not a glossary entry).

Output: `10-discovery/glossary.md`. It is also the binding label source for navigation and every field in the product.

question-generation

Engineers the call's question set: four groups (process, rules, exceptions, consequences) plus a JTBD-style history group where a switch or a past attempt is in play, per-question probing ladders, answerer routing, yield-based prioritization, and a leading-question lint pass.

References: `question-bank.md`: per-silence-class stems, the ladder rungs, the switch/forces questions, workaround excavation (the highest-yield set in mature operations), consequence probes, and the banned forms with their rewrites.

Output: `10-discovery/questions.md`, with a ladder-free share version on top sent to attendees ahead of the call.

interview-facilitation

Runs the call, user interviews and contextual observation: consent, three-column capture, probe technique, live solution separation, requirement-unfolding logging, disagreement handling, and the closing summary.

References: `facilitation-technique.md`: probe mechanics (silence, echo, concretize, show-me, column walk, quantify, ladder up and down, reverse, third party), bias counters for the facilitator's own biases, the seven interviewee response patterns and the move for each, the master-apprentice observation protocol with its five giveaways, and a difficult-moment playbook.

Output: `10-discovery/transcripts/{date}.md` plus the side lists that feed four other skills.

contradiction-mining

Two-stage detection (model sweep with banned verbs, then the human co-satisfiability filter and a deliberate under-detection pass), a seven-type taxonomy, ranking by dependency weight, ownership assignment, and the written-confirmation loop that closes Phase 1.

Output: `10-discovery/problem-statements.md` and the STATE contradictions table.

5.3 Research, knowledge and prototyping PHASE 2A

as-is-mapping

Blueprint-grade current-state capture: trigger, steps with actor and system, the visibility layer with its crossings, data movement, waits, error handling, evidence, and provenance per row. Plus variant mapping, wait-versus-touch accounting, and the pain index.

The discipline: documentary, not aspirational. Do not tidy. A neat as-is is usually a wrong one, and every personal spreadsheet found is an entity the current system failed to model.

incumbent-research

Four extraction targets (data structures, terminology, mental models, failure patterns), the teardown protocol with task walks and severity-rated heuristic findings, state harvesting, implied ERD extraction, source-quality grading A to D, and stop conditions per question.

The discipline: extraction, not admiration. Record complaints as carefully as capabilities; differentiation lives in the complaints. Research never sets a client rule.

knowledge-base

Three-group separation with ranking, same-action indexing, the upgrade scan on authority ingestion, load-bearing marks, conflict marks, citation-mandatory query answering, and the gate-time health audit.

The rule that matters: "the KB cannot answer that" is a correct and complete answer, and it routes the question rather than burying it. Never bridge with general knowledge.

style-guide-authoring

Authors the human-readable style guide and the machine-readable `skills-md/` set that prompts reference instead of adjectives. Seeded from shared-specs where current, with every divergence recorded.

The test every rule must pass: could a reviewer point at a build and say "violates rule X"? If not, rewrite until they could. Files stay under about 150 lines each, one concern per file, with precedence stated in the header, because generators obey short files and skim long ones.

prompt-authoring

The six-part prompt plus the carry checklist, iterated on paper before anyone generates, versioned with a delta log. Also drafts the provisional PRD, structured against the same fourteen audit dimensions so it can be scored like a client spec.

Constraint versus assumption: would violating it make the DA say "that is wrong" (a constraint) or "that is not how I pictured it" (an assumption)? Assumptions are cheap to show and cheap to correct; constraints must hold.

throwaway-prototype

Generates from a complete prompt, enforces the timebox, spends fidelity asymmetrically, frames the build at Gate 1 as an interview instrument, and archives it with its prompt afterwards.

Named failure modes: attachment (the timebox stretches and the throwaway quietly becomes the design) and the approval demo (the gate produces nods instead of corrections, and the phase's cheapest learning window closes unused).

dummy-content

Client-format identifiers with fictitious values, domain-plausible magnitudes, boundary dates, longest-plausible names, at least one negative and one zero per numeric surface, realistic volume, plus adversarial stress content.

Why it is a spec item, not decoration: placeholder text hides column widths, number formatting, truncation, and whether a screen still behaves when a value is negative or zero. Lorem ipsum in a rule-dense screen is the review that did not happen. Never real client data, even "just for the demo".

tentative-structure

The deliberately rough first-pass set: sitemap, to-be sketches, userflow sketches, ERD sketch. All headed TENTATIVE, timeboxed, sources-lined, uncertainty-marked, with an open-questions box per sketch. Includes the group ideation clause, so the model is not the only source of divergence.

Named failure mode: polish creep. The set gets approved instead of corrected.

5.4 Convergence PHASE 2B

requirements-reconciliation

The central artifact. Per-entry reconciliation with mandatory rationale where sources disagreed, the five quality rules, dependency links, supersession, versioning with a change log, the conflict-of-entries scan, and Type B processing from any source at any time.

Named failure modes: restatement (reads like a tidy PRD, shows no disagreement, loses the reason it exists) and rationale rot (corrections applied without why).

erd-modeling

Full entity cards: identity, attributes with sources, cardinality by counterexample, closed status lists via the exhaustion protocol with the terminal check, permissions per action including four-eyes rules, the lifecycle pass, the temporal pass, and the interface-consequences note.

Named failure modes: status leakage (values discovered in Phase 3, each one a missing screen and a Type B), interface-blind lifecycle (the delete button that legally cannot exist), temporal corruption (a price change silently rewriting a year of invoices), and central-noun collapse.

to-be-design

The target process argued against its as-is pair: replacement accounting per pain-marked row, the improvement ladder with the second-job check, automation honesty with four failure-mode answers per automatic step, human decision points with their information needs and service expectations, unhappy paths per step, two-way requirement tracing, and the projected pain-index delta.

process-breakdown

The rule layer developers build against: actor, system, input, output, rule with REQ-ID, exception with handler and resulting state, and downstream effects per step. Plus the three cross-checks (state machine in both directions, coupling pass, permission check) and the two-developer test.

Its quality metric is external: the count of build-time questions the breakdown already answered should approach the count of build-time questions.

ux-flow-optimization

Step accounting (decisions, K/R/N inputs, waits, navigations, recalls with as-is baselines), the six friction classes, the five-rung elimination ladder, the defaults doctrine, progressive disclosure rules, the error-prevention hierarchy, the recovery hierarchy (undo > confirm > warn > nothing), the friction budget sized to consequence, and wait design.

Named failure mode: speeding what should have been deleted. Rung 5 applied where rung 1 was available.

information-architecture

Structural inventory, navigation model selection with rejected alternatives, depth-versus-breadth rules, glossary-bound labelling, wayfinding, cross-cutting placement (settings, admin, reporting, notifications, search, audit, help, bulk), tree-test and card-sort validation, and the route scheme.

The rule that does most work: statuses are not places. Two candidate areas holding the same entity in different states are one area with a filter.

userflows-sitemap

Role-based flows (one per role per primary task), permission-aware branching where a denied role's experience is designed per case, exception joins received explicitly, and the entity-derived sitemap with the feature list used only as a final coverage check.

prioritization

Method selection across seven frameworks with the situations each fits, estimate provenance, the facilitated PO-plus-DA trade session, the mandatory-partition audit against authority classes, the completeness pass, role-coverage and flow-dependency sanity checks, and the signed list.

Named failure mode: must-inflation. MoSCoW without the authority audit, every stakeholder's favourite marked Must, and the method reporting a consensus that does not exist.

5.5 Production intent PHASE 3

hifi-spec

The nine blocks per feature, the state enumeration table built before any screen, the ERD consequences note consumed before designing, REQ-ID citation throughout, and self-reconciliation of status mapping against the ERD before the gate.

visual-design

The hierarchy method (name three levels, spend position then size then weight then colour, squint and five-second tests), the systematic floor, density selection as a domain decision, status encoding, data display rules, and elevation as a z-order language.

References: `type-scale.md` (worked modular scales with line heights, measure, weight, numerals, truncation strategies, accessibility interactions); `color-system.md` (role architecture, ramp construction, the contrast formula with thresholds, status encoding, chart colour, dark mode, forced colours, and a verification procedure); `layout-and-space.md` (the 4px scale, optical adjustment, grids and containers, alignment, composition patterns, page anatomy, concrete density tables, and a review checklist).

component-architecture

Control selection from the shape of the decision, the complete state matrix per component (universal, container, and one per ERD status), composition and slotting rules, the rule of three, naming and API conventions, and the per-component accessibility contract.

References: `component-states.md` (matrices for button, input, select, table, list item, modal, form, status badge, nav item, and the page-level empty/error/permission surfaces); `control-selection.md` (four questions that pick the control, then tables per decision shape).

interaction-design

Feedback timing thresholds, optimistic versus pending versus confirmed with the three eligibility conditions, validation timing, autosave and dirty state, the keyboard contract with focus rules, motion, selection and inline editing and drag, and concurrency handling.

References: `motion-tokens.md` (durations, easings, what may animate, choreography, loading motion, reduced-motion mapping, hard limits); `interaction-patterns.md` (per-pattern decisions a developer will otherwise make alone: buttons, forms, pickers, tables, overlays, notifications, search, dates, files, permissions, session).

responsive-design

The role-by-context table established first (device, environment, input, session length), content-derived breakpoints, the order of collapse, five dense-data strategies with selection criteria, touch and pointer rules, zoom and reflow compliance, density preference, and print/export.

The question it answers first is not "what are the breakpoints" but "who uses this, on what, where, and with what on their hands". Gloves, sunlight, shared terminals and 1366-wide laptops all change the design and none appears in a spec unless asked for.

content-design

Terminology binding to the glossary, voice set concretely, label rules, the four error kinds, the four distinct empty states, confirmation copy sized to consequence, helper text that carries a rule with its REQ-ID, and number/date/unit formatting.

References: `microcopy-patterns.md` (worked examples per surface: errors by kind, the four empties, confirmations by consequence tier, buttons, status transitions, loading, and translation-safe writing).

accessibility

The WCAG 2.2 AA floor as a design-time constraint: seven design-time constraints, per-artifact obligations, and a six-rung testing ladder from keyboard-only through screen reader to testing with disabled users.

References: `wcag-checklist.md`: every level A and AA criterion expressed as what a *design artifact* must contain, plus the AAA criteria worth adopting selectively and when.

Named failure mode: audit-at-the-end, when the tokens, components and flows are all settled and half the findings are unaffordable.

5.6 Quality, gates and close

design-critique

The seven lenses in order (hierarchy, consistency, state completeness, accessibility, content, flow friction, domain fidelity), the fast perceptual tests, four severity levels mapping to Type A and B, the session protocol for running it with people, and the filed report.

Lens 7, domain fidelity, is the one the rest of the design world does not have, and the reason this process exists: does anything on screen contradict a confirmed requirement, offer what the lifecycle forbids, or fail at month-end?

gate-review

Readiness pre-checks, the universal protocol, per-gate specifics, quiet-review handling, classification discipline, client-validation processing, and post-gate verification.

process-evaluation

The standing conformance engine: the traceability matrix, the four sweeps, the conformance snapshot with its headline and metrics strip, and the cadence.

Named failure modes: matrix theatre (a beautiful spreadsheet maintained by hand-invention over uncited artifacts), cadence decay, severity inflation, and evaluator capture (the evaluation run by the same agent whose artifacts it checks).

handoff-package

The full-artifact manifest with gaps listed as gaps, the what-if sweep, the live walkthrough in manifest order, the routing rule, and the question split.

md-spec-loop

The cross-project improvement loop: evidence gathering, divergence adjudication with three outcomes, the build-time question interrogation, dating and versioning, and the three loop metrics.

Why it is worth protecting: it is the only step that pays out on the next project rather than this one, which makes it the easiest to skip. A stale shared file is worse than none, because it produces confidently non-conforming output that looks reviewable and is not.

PART 6

Commands

Nine slash commands. Everything else is conversation with the orchestrator, which routes to agents by matching the task to the routing table.

Command	Does	When
`/new-project {name}`	Copies the workspace template in full, collects the five adaptation variables plus the accessibility obligation and the device-and-environment table, walks the derived obligations checklist, creates risk rows, seeds the conformance scaffolding, then routes into intake.	Day one
`/intake-spec`	Classifies what arrived, audits fourteen dimensions, fills the spec slot, opens the assumption register, starts the substitute path, and discloses a missing spec in STATE. Accepts an argument to re-run on a late-arriving document.	Day one, and whenever a new client document lands
`/phase-status`	Standup: phase and mode, next gate with its unmet exit criteria, blockers, owner-less contradictions, rework-log count by Phase 1 topic, stale artifacts with the dependency that staled them, the conformance headline and its age, and the next three actions with owners.	Any time; start of every session
`/verify [scope]`	Conformance snapshot. Scope is `full` (default), or one of `trace`, `stale`, `provenance`, `drift`, or a REQ-ID or artifact path. Updates the traceability matrix, files the snapshot, routes findings with owners, updates STATE.	Weekly in 2B/3, before gates, after corrections, on demand
`/design-qa [scope]`	The seven-lens critique at the lens set the named gate requires. Runs after `/verify`, not before. Scores and classifies every finding, routes Type B before any fix, runs the perceptual and accessibility tests, files the report.	Before every gate; when a screen feels wrong
`/gate-1`	Attendance and pre-checks, the set reviewed together, the six questions, quiet-review handling, archive, report.	End of Phase 2A
`/gate-2`	Attendance including a developer, model-first review order, ERD acceptance, the signed prioritized list, pain-index deltas presented.	End of Phase 2B
`/gate-3`	Line-by-line against REQ-IDs, independent reconciliation re-run, independent re-run of critique lenses 3, 4 and 7.	End of Phase 3
`/rework "{correction}"`	Classifies Type A or B, routes it, walks dependency links, marks dependents stale, runs the staleness sweep, writes the rework-log row, checks the three-per-topic escalation, and reports the regeneration order.	Whenever feedback arrives outside a gate

PART 7

Artifacts and templates

Sixteen templates in `templates/`. Skills reference them by name, so a template change propagates without touching a skill.

Template	Used by	Carries
`prd-placeholder.md`	spec-intake	The spec slot: header block, the client spec verbatim, or the placeholder record with the fourteen dimensions scored
`assumption-register.md`	spec-intake, everyone	Assumption, why needed, origin, provenance, blast radius, cheapest test, owner, decision date, status, resolution
`open-question.md`	discovery, research	ID, question, raised by, what it blocks, route (RESEARCH / DA-ONLY / PO), status, answer provenance
`problem-statement.md`	contradiction-mining	Problem, source, current workaround, downstream consequence, confidence, confirmation, linked flow rows
`solution-hypothesis.md`	interview-facilitation	Hypothesis, offerer, problems it responds to, status (adopted into REQ / parked / rejected) with reason
`incumbent-product-note.md`	incumbent-research	Source grades, four extraction targets, task walk, refusals, severity-rated heuristic findings, state harvest, implied ERD
`prompt-spec.md`	prompt-authoring	The six parts, the carry checklist, the probe record, and the version delta log
`refined-requirement-entry.md`	requirements-reconciliation	ID, original, correction, resolved, rationale, sources, dependent rule, depends-on/feeds, status, plus the five quality rules
`erd-entity.md`	erd-modeling	Identity, attributes with sources, relationships with counterexamples, status list with transitions and terminal check, permissions, lifecycle, temporal decisions, consequences, open items
`process-step.md`	process-breakdown	Step, actor, system, input, output, rule with REQ-ID, exception with handler, downstream
`ia-map.md`	information-architecture	Structural inventory, navigation model with rejected alternatives, depth map, labelling table, wayfinding, cross-cutting placement, routes
`screen-spec.md`	hifi-spec	The nine blocks, with the state enumeration table first
`design-critique-report.md`	design-critique	Lens set run, findings with location and severity and type and owner, tests run with methods, recurring findings as proposed rule changes
`traceability-matrix.md`	process-evaluation	REQ-ID to artifact-layer mapping, verification method, flags, ghost register, snapshot history
`gate-report.md`	gate-review	Attendance, readiness pre-checks, disclosures, exit criteria, corrections classified, per-gate sections, post-gate verification
`handoff-manifest.md`	handoff-package	Package contents with versions and gaps, the what-if sweep, walkthrough record with the question split, descope pre-answers, the loop review and its three metrics

PART 8

Reference

The dependency order, the guardrails in summary, troubleshooting, extension rules, and a glossary of the system's own vocabulary.

8.1 Artifact dependency order

Used for staleness checks and rework regeneration. `A -> B` means B reads from A, so a change to A stales B.

```
spec-intake → {anchor-doc | derived-brief} → problem-statements → refined-requirements
                                                                → {to-be, erd, breakdown}
spec-intake → assumptions (live; every artifact resting on one names its ASM-ID)
transcripts → problem-statements
problem-statements + open-questions → research-plan → kb
kb → {prd-draft, prompts, style-guide, dummy-content}
prompts + style-guide → prototype
as-is → to-be → flow-optimization → ia → {userflows, sitemap}
erd → {ia, userflows, status-mapping, component state matrices}
erd (statuses) + incumbents (density) → style-guide → skills-md → {prompts, hifi}
refined-requirements + prioritized-list → hifi
hifi → design-critique → gate
shared-specs (.md loop) → prompts + style-guide (next project)
```

> **TWO STALENESS PATHS THAT ARE EASY TO MISS**
>
> A change to the **ERD's status list** stales the style guide's status encoding map, every component state matrix that displays it, and every hi-fi screen state derived from it.
>
> A change to the **IA** stales every userflow, the sitemap, and usually the route scheme.

8.2 Guardrails in summary

`GUARDRAILS.md` is loaded by the orchestrator and every subagent, and overrides any conflicting instruction anywhere else. Twelve sections:

§	Section	Core rule
1	Provenance	Every claim carries a source tag and confidence marker; untaggable claims do not go in
2	The verification ladder	Behaviour-constraining claims require (confirmed) before they can be built against
3	Contradiction handling	Contradictions are deliverables; never silently resolved; unowned ones block gates
4	Prototype containment	Archived after Gate 1; hi-fi never starts from its screens; timebox exceeded is a stop signal
5	Gate integrity	No attendance means NOT HELD; a quiet gate is a warning; sign-off is written
6	Scope discipline	Phase 3 designs only the prioritized list; mandatory items are never deprioritization candidates
7	Model-use discipline	Models draft, humans rule; never present fluency as domain certainty; dummy content is never real
8	Data and confidentiality	Client material stays in the workspace and never enters a web query
9	Escalation triggers	Fourteen named conditions where an agent stops and asks a human
10a	Assumptions and the missing spec	The spec slot always exists; assumptions enter only through the register; absence is disclosed
11	Design floor	The eleven non-negotiables in section 2.9 of this guide
12	Tone of outputs	Artifacts are plain and undecorated; tentative work stays visibly rough; probing questions are asked one at a time

The fourteen escalation triggers

An agent stops and asks a human when: (1) no domain authority is identified and Phase 1 work is requested; (2) an `[LLM]`-only rule is about to enter a design artifact; (3) a gate would be held without required attendance; (4) three or more Type Bs trace to one Phase 1 topic; (5) the prototype timebox is exceeded or hi-fi from the prototype is proposed; (6) a fixed-bid list change is requested after Gate 2; (7) the document and the authority disagree on something a current task depends on, with no owner; (8) shared-specs are more than one project old and unreviewed and a prompt is about to consume them; (9) a behaviour-constraining assumption is about to pass Gate 2 unresolved; (10) a design floor item would be violated or traded away; (11) a design needs a status the ERD lacks or an affordance the lifecycle forbids; (12) Phase 0 is being skipped; (13) the assumption register has not shrunk between two consecutive gates; (14) a gate would be held on a stale conformance snapshot, unresolved blocking findings, or ghost behaviour inside a signed artifact.

8.3 Troubleshooting

Symptom	What it usually means, and what to do
Gate 1 produced almost no corrections	Not a pass. Either the set was too safe or the reviewer was polite. Run the three probes before accepting it, and record that you did. If the set genuinely was too safe, the tentative artifacts were probably too polished.
The same correction keeps coming back at successive gates	The correction was classified Type A when it was Type B, so it was patched in one artifact while the requirement stayed wrong. Re-classify and route through the reconciler; then check whether post-gate verification is being run at all.
Developers ask questions the artifacts already answer	The walkthrough was a survey rather than a depth walk, or the routing rule was stated but not demonstrated. Track the question split; it will show which. Feed the clusters into the loop review as candidate rules.
New statuses keep appearing in Phase 3	The exhaustion protocol was not run with the DA, or was run from memory. Each new status is a Type B and a missing screen. Re-run the status pass on the affected entities before speccing more features.
The assumption register never shrinks	Nobody is chasing answers. This is an escalation trigger in its own right (GUARDRAILS 9.13). Take the overdue entries to the DA as a batch with their blast radii attached.
Every design discussion is about taste	The style guide has aspirations rather than rules. Apply the violation-checkable test to each line and rewrite or delete. A critique with no rules to cite produces only opinions.
The prioritized list has become all Must	The mandatory partition was not audited against authority classes. Only law and contract are automatically mandatory; policy and preference compete. Re-run the audit and re-trade.
The team is answering domain questions from memory	The KB is being bypassed. kb-curator names the pattern; the fix is to route the questions through the KB and let it say "cannot answer", which sends them to open-questions where they belong.
Conformance snapshots keep coming back clean, which feels wrong	Sample an artifact that has not appeared in a finding for two snapshots. Either it is genuinely clean or the sweeps are not really reading it; the sample tells you which.
A client wants to skip Phase 1 because "we know what we want"	Run intake anyway; it takes an hour. The audit scorecard is the conversation: it shows, dimension by dimension, what is actually specified. Four or more ABSENTs is not an argument, it is a measurement.

8.4 Adapting the process to the project

Situation	Adaptation
Small or fast project	`minimum_set: true`. Seven mandatory artifacts; the rest optional but their skills still apply when invoked
Low rule density	ERD collapses to attributes plus relationships; process breakdown optional. Reallocate to UI exploration. But a status machine appearing anywhere makes that entity rule-bearing regardless
Demo engagement	The prototype IS the deliverable: throwaway framing and archive rule off, fidelity table inverts, visual-design and content-design and accessibility apply in full
Modernize engagement	As-is work becomes a product audit; the build gives steps and systems but never waits, workarounds or the spreadsheet beside the keyboard, so observation is still required
Fixed-bid	The list locks at Gate 2; later changes are scope changes requiring escalation before any work
Ready-to-code	The full shared-specs set is a mandatory handoff item; its absence blocks close
Users reachable	Adds user interviews in Phase 1 (separate file, `[USER:role]` tags) and usability testing in Phase 3; contextual observation wherever access allows

8.5 Extending the system

Adding a skill

1. Create `.claude/skills/{name}/SKILL.md` with frontmatter carrying a `description` that states what it does *and when to use it*; that sentence is what triggers it.
2. Follow the standard anatomy (5.1). Keep it under about 300 lines; put reusable banks, tables and numbers in `references/`.
3. Add it to an agent's `skills:` list.
4. If it changes who does what, add a routing row to `CLAUDE.md` section 4.
5. If it produces an artifact, add it to the dependency order (section 5) and to the workspace placeholder for its folder.
6. Add a row to `docs/SKILL-INDEX.md`.

Adding an agent

1. Create `.claude/agents/{name}.md` with `name`, `description` (including when to use it proactively), `tools`, and `skills`.
2. Include all six body sections: goal, operating rules, **handoff contract table**, quality bar, conformance participation, escalations. The handoff table is what keeps the chain working without the orchestrator remembering.
3. Add routing rows to `CLAUDE.md`, and name the seams: what must be settled before this agent starts, and what it must not do that belongs to a neighbour.

Changing a guardrail

Guardrails live in one file, referenced everywhere, so a rule change happens once. Do not copy a guardrail into a skill; cite it by number. If a guardrail needs an exception for a project, that is a `PROJECT_CONFIG` variable with a

routing effect, not a local override.

8.6 Glossary of system terms

Term	Meaning
Anchor document	The document the project treats as its starting statement of requirements. Superseded, never edited, by the refined requirement document at Phase 2B
Assumption register	The live list of what the project is assuming, each with blast radius, cheapest test, owner and decision date
Blast radius	What becomes wrong if an assumption is false: which artifacts, which screens, and which rework class
Consequences note	The ERD's one-line-per-rule translation of lifecycle and permission constraints into interface obligations
Derived brief	The reconstructed specification produced when no spec was supplied. Signed, it becomes the anchor document
Domain authority (DA)	The person who can rule on domain correctness. Without one, Phase 1 cannot converge
Ghost	Artifact behaviour citing no requirement. Either a missing requirement or invention
Orphan	A requirement no artifact implements. Either awaiting its phase, descoped, or forgotten
Pain index	The per-flow rollup (manual movements, uncaught errors, waits, uncommunicated crossings, systems touched) that a to-be must beat
Rule-bearing term	A glossary term whose definition constrains behaviour. Getting it wrong is expensive and silent
Second-job check	Asking what invisible work a step does (control, audit, training, signalling) before eliminating it
Silence	Something the anchor document does not cover. A finding, and the seed of the exceptions question group
Spec slot	`00-inputs/PRD.md`: the client's spec verbatim or the completed placeholder. Never absent
Tentative set	The deliberately rough Phase 2A structure artifacts, built to be corrected at Gate 1
Type A / Type B	Surface correction (patch in place) versus structural correction (route through the reconciler first)
Verification agenda	The anchor analysis output ordering commitments by blast radius, which question generation prioritizes from

WHERE TO GO NEXT

`docs/USAGE.md` is the fifteen-minute operational summary of this guide. `docs/SKILL-INDEX.md` maps all thirty-four skills with their reference files. Every skill file explains its own craft in full, and `GUARDRAILS.md` is short enough to read in one sitting. Everything is plain markdown; nothing is compiled, and nothing hides.